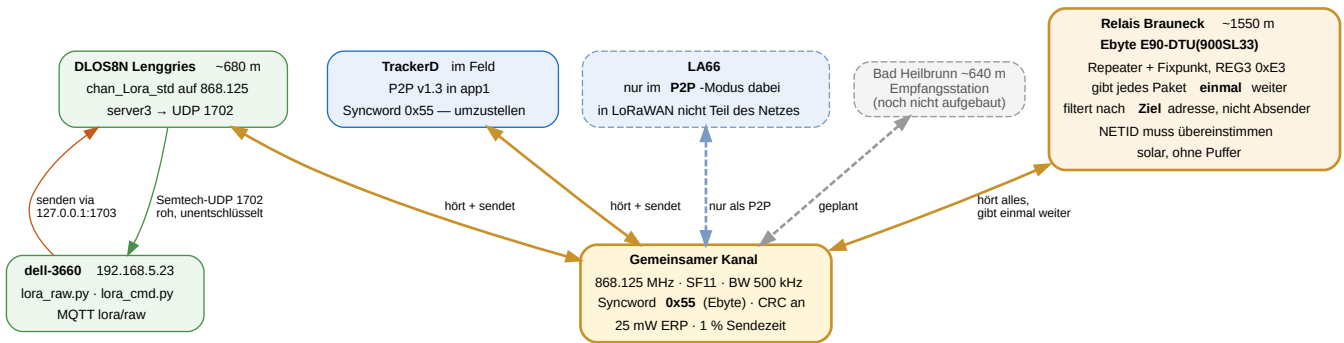


Relaisstelle Brauneck

Krisennetz Lenggries — Aufbau, Betrieb und Befehle. Stand 17.08.2026 · Repository [gerontec/TTN](#)



1 Grundgedanke: ein gemeinsamer Kanal

Alle Teilnehmer arbeiten auf **einer** Frequenz mit **einem** Spreizfaktor. Das ist keine Sparmaßnahme, sondern zwingend: Ein einzelnes Funkmodul kann immer nur auf einem Kanal lauschen. Nur so hört das Relais jeden und erreicht jeden. Es ist dasselbe Flutungsverfahren, das Meshtastic und Ebytes Broadcast-Modus benutzen.

Parameter	Wert	Warum
Frequenz	868.125 MHz	Ebyte-Kanal 18 ab Werk (850.125 + 18); liegt im Band 868.0–868.6
Spreizfaktor	SF11	Ebyte-Luftrate 2.4k; die Etiketten sind nominal, gemessen ist SF11
Bandbreite	500 kHz	Die Ebyte-Leiter ist durchgehend BW500, nie BW125 — nachgemessen
LDRO	1	Ebyte sendet mit 1; die Rechenregel käme bei SF11/BW500 auf 0 und jede Nutzlast bekäme CRC-Fehler
Syncword	0x34 (öffentlich)	Historisch, weil der Rohkanal anfangs das Syncword der LoRaWAN-Kanäle mitbenutzen musste. Seit 17.08.2026 nicht mehr nötig — siehe Kapitel 8.
CRC	an	sonst verwirft der Paket-Forwarder
Leistung	bis 14 dBm ERP	25 mW ist das Limit in 868.0–868.6
Sendezeit	1 %	bei ~144 ms Luftzeit rund 14 s Sperre je Paket — der eigentliche Engpass, siehe Kapitel 6

Streckenrechnung 10 km, Sichtverbindung. 14 dBm + 2 dBi = 16 dBm EIRP, minus 111 dB Freiraumdämpfung, plus 2 dBi Empfangsantenne ergibt –93 dBm. Gegen rund –128 dBm Empfindlichkeit bei SF11/BW500 bleiben etwa **35 dB Reserve** — SF11 auf 500 kHz ist trotz der breiteren Bandbreite noch etwa 4 dB empfindlicher als das frühere SF7/BW125. Den Sprung ins Nachbartal trägt der Bergstandort (~1550 m gegen zwei Täler auf ~650 m), nicht die Sendeleistung.

2 Was das Relais tut

Es hört alles auf dem gemeinsamen Kanal und gibt jedes Paket **genau einmal** weiter, versehen mit einem Sprungzähler. Damit erreicht ein Knoten, dessen Direktsignal zu schwach wäre, das Tal trotzdem.

Nutzlast beginnt mit	Bedeutung	Verhalten
C>	Fernwirkbefehl	ausgeführt, Antwort gesendet, nie weitergegeben
A>	Antwort einer Station	ignoriert
R<n>>	bereits n-mal weitergegeben	weiter bis 3 Sprünge, danach verworfen
alles Übrige	frisches Paket	als R1>... weitergegeben

Eigenecho

Da alle denselben Kanal teilen, trägt allein die Marker-Logik:

- Während des Sendens ist der Empfänger taub — die eigene Aussendung hört die Station nie unmittelbar.
- Jedes weitergegebene Paket bekommt R<sprung>>. Kommt es über eine zweite Relaisstelle zurück, greift der Zähler.
- Der Dublettenspeicher schlüsselt auf den Inhalt **ohne** Marker und sperrt denselben Text fünf Minuten.

Mehrere Relaisstellen sind dadurch möglich: aus R1> wird R2> und so fort, bis drei Sprünge erreicht sind.

3 Fernwirken von 192.168.5.23

Der Pico hat **kein WLAN** — es ist ein RP2040, kein Pico W; das Modul `network` existiert nicht. Auf dem Berg gibt es weder SSH noch OTA. Der einzige Rückkanal ist der Funk, auf dem das Relais ohnehin arbeitet. Für ein Krisensystem ist das der richtige Weg: Er trägt genau dann, wenn alles andere ausgefallen ist.

```
python3 /home/gh/python/lora_cmd.py POWER 17
gesendet: C>POWER 17
Antwort: POWER 17 dBm (RSSI -101, SNR 8.8)
```

Befehl	Wirkung
POWER 2..22	Sendeleistung in dBm — der wichtigste Befehl
STATUS	weitergegeben / unterdrückt / Laufzeit / Konfiguration
RELAY 0 1	Weitergabe aus- oder einschalten
TELEM 0 1	Quittungen aus- oder einschalten
SAVE	Konfiguration ausdrücklich sichern
REBOOT	Neustart des Boards
PING	lebt die Station?

Frequenz und Spreizfaktor sind bewusst nicht fernstellbar. In einem Einkanalnetz würde man sich damit den Ast absägen, auf dem man sitzt — die Station wäre nach dem Wechsel unerreichbar.

Ohne Authentisierung. Wer in Funkreichweite ist, kann das Relais umstellen. Bewusste Abwägung zugunsten der Einfachheit.

Weg des Befehls: `lora-raw.service` hält UDP 1702 dauerhaft und ist der einzige, der senden kann. Er hat deshalb einen Steuereingang auf `127.0.0.1:1703` ; was dort ankommt, wird beim nächsten `PULL_DATA` des Gateways gefunkt. `lora_cmd.py` schickt den Befehl dorthin und wartet über MQTT `lora/raw` auf die Antwort.

4 Betriebsart umschalten

TrackerD — LoRaWAN ↔ P2P

Der TrackerD hat zwei Firmware-Slots: **app0** trägt die Dragino-LoRaWAN-Firmware, **app1** die selbst gebaute P2P-Firmware. Umgeschaltet wird ausschließlich über `otadata` — `app0`, `app1` und vor allem das NVS mit den LoRaWAN-Keys bleiben unangetastet. Der Wechsel ist daher verlustfrei und beliebig wiederholbar.

Richtung	Befehl	gemessen
LoRaWAN → P2P	<code>switch_app.py p2p --port /dev/ttyACM1</code>	1,35 s
P2P → LoRaWAN	<code>AT+LORAWAN</code> — am Gerät, ohne PC-Werkzeug	2,24 s inkl. Neustart
P2P → LoRaWAN	<code>switch_app.py lorawan --port /dev/ttyACM1</code>	gleichwertig
Zustand abfragen	<code>switch_app.py status --port /dev/ttyACM1</code>	liest <code>otadata</code>
P2P neu bauen	<code>pio run</code> , dann <code>switch_app.py flash</code>	schreibt nur <code>app1</code>

Niemals `pio run -t upload`. Das würde Partitionstabelle und `app0` überschreiben und die LoRaWAN-Firmware samt Keys zerstören. Geflasht wird gezielt nach `0x1F0000` durch `switch_app.py`.

LA66 — LoRaWAN ↔ P2P

```
python3 la66_mode.py status      # Modus am Boot-Banner erkennen
python3 la66_mode.py p2p         # 37640 B, 5,0 s
python3 la66_mode.py lorawan     # 69032 B, 9,1 s
```

Beim LA66 wird wirklich geflasht, nicht umgeschaltet: Beide Dragino-Images sind fest auf `0x0800D000` gelinkt, ein zweiter Slot ist unmöglich. Ein Flash setzt die Funkparameter auf Werk zurück — dafür `--apply-rf`. Die DevEUI überlebt.

Im LoRaWAN-Modus ist ein Gerät nicht Teil dieses Netzes. Nachgemessen: Der LA66 sendet mit `AT+DR=0` auf SF12 und springt über alle acht EU868-Kanäle (beobachtet: 867.3 und 868.3 MHz). Der Pico auf 868.125 hörte in 95 s **null** Pakete (damals auf SF7; mit dem heutigen SF11/BW500 gilt es unverändert). Spreizfaktoren sind quasi-orthogonal — ein Empfänger mit festem SF kann einen anderen nicht demodulieren.

Daraus folgt grundsätzlich: **Ein Knoten mit einem Funkmodul kann kein LoRaWAN-Relais sein.** LoRaWAN wechselt pro Sendung den Kanal und passt per ADR den Spreizfaktor an; ein Empfänger mit fester Frequenz und festem SF kann dem nicht folgen. Genau dafür hat das Gateway acht parallele Demodulatoren. Hinzu kommt: Der Marker `R1>` würde die MIC-Prüfung eines LoRaWAN-Rahmens zerstören.

5 Stromversorgung: reiner Solarbetrieb

Der Victron-Laderegler schaltet den Verbraucher morgens schlagartig zu und abends ab. Eine Pufferbatterie gibt es nicht. Das prägt den Betrieb stärker als jede Funkeinstellung.

- **Jeder Morgen ist ein Kaltstart, und niemand ist oben.** `main.py` startet das Relais, wiederholt bei Fehlern und löst notfalls `machine.reset()` aus, statt in den REPL zu fallen.
- **Konfiguration sichert sich sofort**, nicht erst auf `SAVE` — eine per Funk gesetzte Sendeleistung wäre sonst am nächsten Morgen weg.
- **Beschädigte `relais.json` wird abgefangen**; die Station kommt dann mit Vorgabewerten hoch, aber sie kommt hoch.
- **Nachts ist die Kette unterbrochen.** Eigenschaft des Aufbaus, keine Störung — wer nachts Reichweite braucht, braucht einen Akku.

Systemtakt 48 MHz

125 MHz sind sinnlos: Modulation, Timing und Preamble macht der SX1262 selbst, der Prozessor wartet nur. Weniger Takt heißt weniger Strom, und bei einer Versorgung ohne Puffer weniger Spannungseinbruch unter Last.

Takt	Ergebnis der Abtastung
125 / 96 / 64 / 48 / 32 / 24 MHz	sauber — <code>DevErr 0x0000</code> , Syncword <code>3444</code> , TX ok
18 MHz	SPI liefert Müll — Syncword liest <code>a2a2</code> , TX schlägt fehl
12 MHz	<code>machine.freq()</code> lehnt ab

Gewählt sind **48 MHz**: reichlich Abstand zur Ausfallgrenze bei gut halbiertem Prozessorstrom. Der Takt wird vor dem Aufsetzen des Funkchips gestellt, weil `clk_peri` am Systemtakt hängt.

6 Nachgewiesen über die Luft

Alle drei Stationen in einem Durchlauf, nach dem Kaltstart der neuen Firmware und **ohne** manuelles Nachsetzen des Syncwords:

```
TrackerD sendet  "V13-KALTSTART-1"
Pico            weiter: Sprung 1 RSSI -17 SNR 12.0 20 dBm, 51 ms
Gateway hört    RSSI -83 "V13-KALTSTART-1"      <- direkt
Gateway hört    RSSI -84 "R1>V13-KALTSTART-1"   <- über das Relais
Pico-Bilanz     2 gehört, 2 weitergegeben, 0 unterdrückt
```

In einem früheren Lauf war der Gewinn deutlicher messbar: Original -92 dBm, Kopie vom Relais -73 dBm — **19 dB stärker**. Genau dafür steht die Station auf dem Berg. Der TrackerD empfing dabei sein eigenes Paket als `R1>...` zurück; das Relais gab es **nicht** erneut weiter.

Ebyte-Broadcasts, und was sie kosten

Ein Ebyte-Rahmen wird **unverändert** weitergereicht — kein `R1>` davor. Er trägt einen eigenen 8-Byte-Kopf, und ein Empfänger liest die ersten acht Byte als eben diesen; ein vorangestelltes „R“ macht den Rahmen unlesbar. Gegen Schleifen trägt hier allein der Dublettenspeicher, einen Sprungzähler gibt es in diesem Format nicht.

Am Gateway ist beides zu sehen, unterscheidbar am Quarzversatz:

```
foff -27673 RSSI -76 Original vom E22
foff -144 RSSI -88 Kopie vom Relais (byteweise identisch)
```

Das Sendezeitbudget ist der Engpass, nicht die Empfindlichkeit. Bei SF11/BW500 dauert ein 16-Byte-Rahmen 144 ms; die 1%-Regel sperrt danach rund 14 s. Vier Broadcasts im Abstand von 3,5 s:

```
weiter: Ebyte, unverändert RSSI -23 SNR 8.3 144 ms
verworfen: Sendezeitbudget, noch 11.0 s gesperrt
verworfen: Sendezeitbudget, noch 7.5 s gesperrt
verworfen: Sendezeitbudget, noch 4.0 s gesperrt
Bilanz: 4 gehoert, 1 weitergegeben, 3 unterdrueckt
```

Drei von vier fielen nicht am Funk aus, sondern an der Rechtslage. Mit dem früheren SF7/BW125 waren es 72 ms und rund 7 s — der Umstieg auf das Ebyte-Profil hat das verdoppelt. Wer auf Durchsatz auslegt, muss hier ansetzen: kürzere Nutzlasten, seltener senden, oder ein zweites Modul für das Band 869.4–869.65 MHz mit 10 % Sendezeit.

7 Dateien

Ort	Datei	Zweck
Pico	main.py	Selbststart nach jedem Sonnenaufgang
	repeater.py	Flutung, Sprungzähler, Dubletten, Sendezeitbudget
	fernwerk.py	Befehle, Konfiguration in /relais.json
	lora_p2p.py	SX1262-Treiber
dell 192.168.5.23	lora_raw.py	Roh-Abgriff UDP 1702, Steuereingang 1703, MQTT
	lora_cmd.py	Fernwirkbefehle absetzen
TrackerD	p2p/src/main.cpp	P2P-Firmware v1.3, Syncword 0x34 ab Werk
	switch_app.py	otadata-Umschaltung, Flashen nach app1

8 Anleitung: Ebyte-Rohkanal am eigenen Gateway

Diese Anleitung richtet sich an andere Besitzer eines Gateways mit **SX1302** (Dragino DLOS8N, LPS8v2, RAK7268 und Verwandte), die ein Ebyte-Modul — E22, E90-DTU — neben dem laufenden LoRaWAN-Betrieb empfangen wollen. Alle Werte stammen aus eigenen Messungen, nicht aus Datenblättern.

Der Kern in einem Satz. Der SX1302 hat **vier** RX-Syncword-Registerpaare, nicht eines: drei für den gemeinsamen MultiSF-Block (getrennt nach SF5, SF6 und SF7–12, gültig für alle acht LoRaWAN-Kanäle) und **ein eigenes** für den LoRa-Service-Modem hinter `chan_Lora_std`. Der Rohkanal kann deshalb ein beliebiges Syncword tragen, *während* die acht LoRaWAN-Kanäle unverändert auf 0x34 bleiben. Nur schreibt der Semtech-HAL dort stur 0x12 oder 0x34, abgeleitet aus `lorawan_public`.

8.1 Was nicht geht

Einer der acht MultiSF-Kanäle kann **kein** eigenes Syncword bekommen. Die acht teilen sich einen Demodulator-Block, dessen Syncword nur nach Spreizfaktor aufgeteilt ist; ein Register pro IF-Kanal existiert im Chip nicht. Auch Zeitmultiplex hilft nicht: Umschalten kostet zwar nur 590 µs hin und zurück, aber es gibt keinen Auslöser — die Paketerkennung *ist* der Syncword-Vergleich, und wenn ein Paket auffällt, ist es längst verworfen.

8.2 Ebyte-Modul auslesen

Das Modul muss im Konfigurationsmodus stehen ($M0 = 1$, $M1 = 1$; bei USB-Adaptern oft über DTR/RTS). Dann liefert `C1 00 09` die neun Konfigurationsbytes:

```
C1 00 09 | FF FF 00 62 00 12 80 00 00
          ADDH ADDL NETID REG0 REG1 REG2 REG3 CRYPT_H CRYPT_L
```

Feld	Beispiel	Bedeutung
REG2	0x12 = 18	Kanal → Frequenz = 850.125 MHz + Kanal × 1 MHz = 868.125 MHz (900-MHz-Reihe)
REG0 Bits 2–0	2	Lufrate, hier 2.4 k
REG0 Bits 7–5	3	UART-Baudrate, hier 9600
REG3 Bit 6	0	0 = transparent, 1 = Festpunkt mit Adresskopf

Die Lufraten-Etiketten sind nominal — nicht rechnen, messen. Die im Netz kursierende Tabelle übersetzt sie nach BW 125 kHz. Das ist falsch. Gemessen ist die Ebyte-Leiter durchgehend **BW 500 kHz**: Index 2 („2.4k“) ist **SF11/BW500**, Index 5 („19.2k“) ist SF7/BW500. Das Handbuch bestätigt es beiläufig mit „air data rate 2.4kbps@SF11“ — 2,4 kbps bei SF11 gehen rechnerisch nur mit BW 500 auf, bei BW 125 wären es 537 bps.

8.3 Rohkanal am Gateway einstellen

`chan_Lora_std` ist der neunte, eigenständige Kanal. Er wird frei in Frequenz, Spreizfaktor und Bandbreite gesetzt:

```
"chan_Lora_std": {"enable": true, "radio": 1, "if": -375000,
                  "bandwidth": 500000, "spread_factor": 11, ...}
```

Die Frequenz ergibt sich als `radio[n].freq + if`; für 868.125 MHz bei `radio1_freq = 868.5 MHz` also `if = -375000`.

Falle: die Datei, die man editiert, ist die falsche. `init_board()` in `/etc/init.d/lora_gw` ruft bei jedem Start `/usr/bin/generate-config.sh`, und das **kopiert** `/etc/lora/cfg-302/EU-global_conf.json` über `/etc/lora/global_conf.json`. Änderungen an der zweiten Datei sind nach dem nächsten Neustart spurlos weg. Zu ändern ist die **Vorlage**.

8.4 Syncword freischalten

Das Werkzeug liegt im Repository unter `gateway/sx1302_syncword/`. Es ersetzt per `LD_PRELOAD` genau eine HAL-Funktion, `sx1302_lora_syncword()`, deren Signatur keine Structs enthält und daher ABI-neutral ist. Die HAL selbst wird nicht ausgetauscht — bei Dragino steckt der Chip-Reset in `libsx1302hal.so`, ein Upstream-Build würde ihn verlieren.

```
# Cross-Build, OpenWrt-18.06-SDK, mips_24kc, musl
make SDK=/pfad/zu/openwrt-sdk-18.06.9-ar71xx-generic_gcc-7.3.0_musl.Linux-x86_64
make install GW=root@<gateway>
```

`/etc/lora/syncword.conf`:

```
sf5      = auto      # auto = unverändertes Verhalten aus lorawan_public
sf6      = auto
sf7to12  = auto      # 0x34 -> LoRaWAN bleibt unberührt
service  = 0x55      # chan_Lora_std allein: Ebyte-Werkswert
ldro     = 1         # bei BW500 zwingend, siehe unten
```

Aktiviert wird der Shim über den Wrapper `/usr/bin/fwd_syncword`, **nicht** über `procd_set_param env`: `procd` setzt für die Zeilenpufferung selbst `LD_PRELOAD=/lib/libsetlbf.so` und würde einen so gesetzten Wert überschreiben. Der Wrapper hängt den Shim mit `:` getrennt *nach* `procd` an.

LDRO muss von Hand auf 1. Der HAL leitet es aus `SET_PPM_0N(bw, dr)` ab, das nur bei BW125 mit SF11/SF12 und BW250 mit SF12 wahr wird — bei **BW500 also nie**. Ebyte sendet aber mit LDRO 1. Bei falschem LDRO rastet der Header sauber ein, und *jede* Nutzlast kommt mit CRC-Fehler an. Das sieht aus wie „fast richtig“ und ist die zeitraubendste Falle der ganzen Strecke.

8.5 Das Ebyte-Syncword ist 0x55

Ebyte lässt das Syncword nicht konfigurieren; es ist ab Werk verdrahtet und steht in keinem Handbuch. An einem SX126x-Empfänger lässt es sich **nicht vollständig** bestimmen: Der wertet nur das erste der beiden Syncword-Registerbytes aus, weshalb dort alle acht Werte 0x58–0x5F treffen.

Der SX1302 prüft **beide** Peak-Positionen streng und klärt es deshalb. Ein Syncword `0xHL` steckt in den zwei Sync-Symbolen der Präambel; der SX1302 speichert je Symbol `symbolwert / 4`, also **peak_pos = nibble × 2**. Das Feld ist 5 Bit breit und wird vorzeichenlos maskiert — alle 256 Werte sind erreichbar, die Beschränkung auf 0x12/0x34 ist reine Softwarekonvention.

Register	Adresse	gilt für
SF5_PEAk1/2	0x588A/0x588B	alle 8 MultiSF-Kanäle, nur SF5
SF6_PEAk1/2	0x588C/0x588D	alle 8 MultiSF-Kanäle, nur SF6
SF7T012_PEAk1/2	0x588E/0x588F	alle 8 MultiSF-Kanäle, SF7–12
LORA_SERVICE_PEAk1/2	0x5B2E/0x5B2F	nur chan_Lora_std

8.6 Wenn nichts ankommt: peak2 absuchen

Fremde Module können ein anderes unteres Nibble benutzen. Statt den Forwarder sechzehnmal neu zu starten, wird das Register im laufenden Betrieb gesetzt — der SX1302 hat kein Page-Register, jeder Zugriff ist ein einzelnes `ioctl` und wird vom Kernel gegen den Forwarder serialisiert:

```
for n in $(seq 0 15); do
    sx1302_poke 0x5B2F $((n*2))
    vorher=$(grep -c '"chan":8' /tmp/fwd.log); sleep 3
    nachher=$(grep -c '"chan":8' /tmp/fwd.log)
    printf 'sw=0x5%X peak2=%2d neue Pakete: %d\n' $n $((n*2)) $((nachher-vorher))
done
```

Das Sendegerät muss dabei durchgehend funken. Genau so wurde 0x55 gefunden: Pakete ausschließlich bei `peak2 = 10`.

8.7 Prüfen

```
logread | grep syncword
[syncword] LoRa Service LDR0 forced to 1 (HAL rule overridden)
[syncword] multi-SF ch0-7: SF5=0x12 SF6=0x12 SF7-12=0x34 | Lora_std (SF11): 0x55

sx1302_poke 0x5B2E -> 0x0A peak1, oberes Nibble 5
sx1302_poke 0x5B2F -> 0x0A peak2, unteres Nibble 5
sx1302_poke 0x5B22 -> 0x98 Bits 4-5 = 1, LDR0 aktiv
```

Ein empfangenes Paket erscheint als `rxpk` auf **chan 8**:

```
{"chan":8,"freq":868.125000,"datr":"SF11BW500","rssi":-63,"stat":1,
 "size":15,"data":"LBKHJgD//wdCQF1WPyIh"}
```

8.8 Das Ebyte-Rahmenformat

Auch im transparenten Modus verpackt das Modul die Nutzlast. Der Kopf ist acht Byte lang, und die Nutzlast ist mit der **Kanalnummer** XOR-verweift — dieselbe Zahl steht in Byte 1:

```
2C 12 87 26 00 FF FF 07 | 42 40 5D 56 3F 22 21
                        XOR 0x12  -> "PROD-03"
```

Byte 0 0x2C Kennung
Byte 1 Kanalnummer
Byte 2-3 xx, xx ^ 0xA1 mit xx = (XOR über alle Nutzlastbytes) ^ 0xA0
Byte 4 NETID
Byte 5-6 Zieladresse des Rahmens (FF FF = Rundruf)
Byte 7 Länge der Nutzlast

Byte 2–3 sind eine **Prüfsumme, kein Zähler** — gleiche Nutzlast ergibt einen Byte für Byte identischen Rahmen. Und der XOR-Schlüssel ist konstant `0x12`, nicht die Kanalnummer; beim 868er fällt beides zufällig zusammen (Kanal 18 = `0x12`), der 433er sendet auf Kanal 23 und weißt trotzdem mit `0x12`. Byte 5–6 tragen die **Zieladresse**, nicht den Absender — im Beispiel `FF FF`, also einen Rundruf; ein Absender steht im Rahmen überhaupt nicht. Zusammen mit Byte 4 sind das die beiden Felder, aus denen sich das Gruppenkonzept in Kapitel 9 ergibt.

8.9 Was der Eingriff nicht anfasst

Nachgemessen mit einem LA66-USB (EU868 v1.3, bereits gejoint): Uplink auf 868.500 MHz bei DR0 kommt unverändert auf `chan 2 an, SF12BW125, stat:1`, Rahmen sauber — MHDR `0x40`, DevAddr, FPort 2, Nutzlast lesbar. **Der LoRaWAN-Betrieb bleibt vollständig erhalten**, weil ausschließlich das Register des Service-Modems verändert wird.

8.10 Alles in einem Skript

Die fünf Handgriffe — Vorlage, `syncword.conf`, Wrapper, Init-Skript, Neustart — fasst `gateway/sx1302_syncword/setup_ebyte_rawchannel.sh` zusammen. Es läuft auf dem Gateway, sichert jede angefasste Datei nach `.pre-ebyte` und prüft sich am Ende selbst:

```
setup_ebyte_rawchannel.sh --apply        # einrichten und pruefen
setup_ebyte_rawchannel.sh --status       # Ist-Zustand inkl. Registern
setup_ebyte_rawchannel.sh --revert       # alles zurueck, Stock-Verhalten
```

Vorgaben passen zu einem Ebyte auf Werkskonfiguration; abweichend etwa `--sf 7 --bw 500000 --sync 0x55` für Lufrate 19.2k. Ausgabe eines erfolgreichen Laufs:

```
==> Vorlage angepasst (Sicherung: ../EU-global_conf.json.pre-ebyte)
==> Init-Skript angepasst (Sicherung: /etc/init.d/lora_gw.pre-ebyte)
[syncword] LoRa Service LDR0 forced to 1 (HAL rule overridden)
[syncword] multi-SF ch0-7: SF5=0x12 SF6=0x12 SF7-12=0x34 | Lora_std (SF11): 0x55
      0x5B2E = 0x0A (10) peak1
      0x5B2F = 0x0A (10) peak2
```

8.11 Die Gegenrichtung: Gateway sendet

Alles oben betrifft den **Empfang**. Soll das Gateway den Ebyte-Knoten auch *erreichen* — etwa einen E90-DTU-Repeater auf dem Berg —, greift eine andere Stelle im HAL: `sx1302_send()` schreibt das Sende-Syncword **für jedes Paket neu**, unmittelbar vor dem Tasten, wieder abgeleitet aus `lorawan_public`. Ein Downlink geht damit stets als `0x34` hinaus, und eine Gegenstelle auf `0x55` hört ihn nie.

Der Shim kann auch das (`tx = 0x55` in `syncword.conf`); umgesetzt ist es durch Interposition von `lgw_com_rmw()` mit Umschreiben der vier TX-Register `0x526D/0x526E` und `0x546D/0x546E` — adressbasiert, also wieder ohne Bindung an Draginos Strukturen.

`tx` allein würde auf *jeden* Downlink wirken, auch auf die von LoRaWAN — Join-Accepts und ADR gingen dann auf einem Syncword hinaus, das kein Endgerät annimmt. Deshalb gibt es `tx_freq` :

```
tx          = 0x55
tx_freq = 868125000      # nur Downlinks auf dieser Frequenz, Fenster +/- 5 kHz
```

Möglich wird das dadurch, dass `sx1302_send()` die Sendefrequenz in `loragw_sx1302.c:2604-2608` schreibt, **bevor** es in `:2710` zum Syncword kommt. Der Shim liest die drei Frequenzbytes im Vorbeigehen mit (sie laufen als 8-Bit-Direktschreibung über `lgw_com_w()` , nicht über `lgw_com_rmw()`) und entscheidet dann je Sendekette. Die Auflösung beträgt $32\text{ MHz} / 2^{18} = 122\text{ Hz}$, das trennt den Rohkanal auf 868.125 mühelos von `chan_multisf_0` auf 868.100, 25 kHz daneben.

tx_freq	Downlink auf 868.125	peak1 / peak2	Syncword
868125000	Rohkanal getroffen	10 / 10	0x55 umgeschrieben
869525000 (RX2)	passt nicht	6 / 8	0x34, HAL-Wert unberührt

Beim Ablesen der Register nicht stolpern: das Peak-Feld sind die **unteren fünf Bit**. `0x526D = 0xAA` heißt `0xAA & 0x1F = 10` ; die oberen Bits tragen `AUTO_SCALE`, `GAIN` und `DROP_ON_SYNC`.

Funkrechtlich. 868.0–868.6 MHz erlaubt 25 mW ERP bei 1 % Sendezeit. Ein Rohkanal mit BW 500 kHz auf 868.125 MHz belegt 867.875–868.375 MHz und überlappt damit die LoRaWAN-Kanäle 868.1 und 868.3 — im selben Unterband zulässig, aber beim Sendezeitbudget mitzurechnen.

9 Gruppenkonzept: zwei Netze, eine Brücke

Das Netz trennt seit dem 17.08.2026 zwei Gerätegruppen, verbunden allein durch den E90-DTU als Relais. Dahinter stehen **zwei voneinander unabhängige Mechanismen**, die sich leicht verwechseln lassen — ich habe sie beim Erarbeiten mehrfach vermischt.

	Feld	Quelle
Adressierung	Kanal + Zieladresse	Handbuch T22U-Serie, 4.1/4.2
Weiterleitung	NETID-Paar	Handbuch, 5.3 Relay Networking

9.1 Wie Ebyte adressiert

```
00 03 | 04 | AA BB CC
Ziel-   Ziel- Daten
adresse kanal
```

Der **Kanal grenzt die Gruppe ab**, die **Adresse wählt ein Mitglied** darin. Bei Broadcast `FF FF` geben alle Module auf dem Kanal aus — eines auf einem anderen Kanal schweigt auch dann.

Die Adresse im Rahmen ist das Ziel, nicht der Absender. Die eigene Adresse des Senders taucht im Paket überhaupt nicht auf. Ein Ebyte-Rahmen sagt also, *an wen* er geht — nie, *von wem* er kam. Frühere Fassungen dieses Handbuchs und von `dell/loraw_raw.py` beschrifteten Byte 5–6 als Absender; das war falsch.

9.2 Warum eine Gruppe über Kanäle nicht geht

Wäre der Kanal die Gruppentrennung, kostete jede Gruppe eine eigene Frequenz: `850.125 MHz + Kanal`. Kanal 18 ist unsere 868.125 MHz, Kanal 19 wäre **869.125 MHz** — außerhalb von 868.0–868.6. Im nutzbaren europäischen Unterband ist für eine zweite Gruppe schlicht kein Platz.

9.3 Deshalb: NETID als Gruppenwähler

Kanal 18 (868.125 MHz)	gemeinsame Funkgruppe
Adresse 2201	Netzschlüssel aller Knoten, bewusst kein Broadcast
└─ NETID 00 Gruppe A	zehn E22, dell
└─ NETID BB Gruppe B	zehn E22
E90-Relais ADDH=00 ADDL=BB	einzige Brücke, bidirektional
Gateway	ohne NETID, hört alle Gruppen

Die Adresse muss 2201 sein und darf nicht FFFF sein. Das Handbuch legt die Rangfolge fest: „*Network code filtering has lower priority than broadcast addresses. Even with differing network codes, broadcast data can still be received.*“ Mit Broadcast wären die Gruppen also durchlässig — die NETID filtert nur, solange die Adresse keine Broadcast-Adresse ist. Die Adresse dient hier deshalb als gemeinsamer Netzschlüssel, die Trennung leistet allein die NETID.

9.4 Zwei Gruppen zu je zehn Knoten

Zwanzig E22 verteilen sich auf die beiden Netze. Unterschieden werden sie **allein** durch die NETID — alles Übrige ist über beide Gruppen identisch, sonst hört sich niemand.

Register	Gruppe A	Gruppe B	Bedeutung
NETID (0x02)	00	bb	Gruppenwähler
ADDH/ADDL	22 01	22 01	Netzschlüssel, kein Knotenname
REG0	62	62	9600 8N1, Lufrate 2.4k
REG2	12	12	Kanal 18 → 868.125 MHz

Alle zwanzig tragen dieselbe Adresse. Sie ist kein Gerätename, sondern der gemeinsame Netzschlüssel; eine Knotenkennung gehört in die **Nutzlast**, nicht ins Adressfeld. Einzeladressierung scheidet im Transparentmodus ohnehin aus, siehe 9.8.

Nie FFFF ausrollen. Broadcast hat Vorrang vor der Netzkennung — mit FFFF fielen die beiden Gruppen zu einer zusammen, und jeder Rahmen käme zusätzlich als Relaisdublette an. Der Netzschlüssel ist damit nicht Zierrat, sondern die Bedingung dafür, dass die NETID überhaupt filtert.

Querverkehr läuft ausschließlich über den Berg. Ein Rahmen aus Gruppe A trägt NETID 00 ; ein B-Knoten verwirft ihn, auch wenn er ihn mühelos direkt empfängt — erst die Relaiskopie mit NETID bb kommt bei ihm an. Zwei Folgen: Querverkehr belegt die Luft doppelt, und er endet abends mit dem Relais. Die Solarversorgung ohne Puffer aus Kapitel 5 ist damit zugleich die Betriebszeit der Gruppenbrücke. Innerhalb einer Gruppe bleibt davon alles unberührt.

Werksvorgabe eines E22 ist NETID 00 und Adresse 0000 — ein fabrikfrisches oder zurückgesetztes Modul liegt damit formal in Gruppe A. Mitlesen kann es nicht (0000 ist nicht der Netzschlüssel), und seine Aussendungen nimmt niemand an; es kostet allein Sendezeit. Auf Gruppe B kann der Fall gar nicht eintreten.

Ausgerollt wird mit `python/e22_group_setup.py` : Modul in den Konfigurationsmodus (E22 M0=1, M1=1), anstecken, `--group A` oder `--group B` . Das Skript schreibt den vollständigen Registerblock, liest ihn zurück, vergleicht byteweise und protokolliert jeden Lauf in `e22_rollout.log` . Hängt am Port ein Modul mit gesetztem Relay-Bit, verweigert es die Arbeit — sonst überschreibt der erste Rollout das Relais und nimmt die Bergstation aus dem Netz.

9.5 Die Weiterleitungsregel

Im Relaismodus sind ADDH / ADDL keine Adressen mehr, sondern das **NETID-Paar**: „*If data is received from one network, it is forwarded to the other network.*“ Der E90 steht auf ADDH=00, ADDL=BB und überträgt bidirektional zwischen diesen beiden — und nur zwischen ihnen.

Er ändert dabei genau **ein** Byte:

2c 12 68 c9 00 22 01 03 ...	Original
2c 12 68 c9 bb 22 01 03 ...	Weitergabe

Prüfbyte, Zieladresse, Länge und Nutzlast bleiben unangetastet — nur die NETID wird auf die Gegengruppe gesetzt, sonst verwürfen deren Empfänger den Rahmen.

Vorher stand der E90 auf `ADDH = ADDL = 0x00`, leitete also von NETID 0 nach NETID 0 zurück. Ebyte warnt davor ausdrücklich: „Using two or more relays with **identical** ADDH and ADDL addresses is not recommended, as it may cause **circular forwarding**.“ Das erklärte das Echo, das der E22 auf seiner seriellen Seite sah — ein Symptom, kein Merkmal.

9.6 Nachgewiesen

Gleiche Adresse, gleicher Kanal, gleiche Modulation — nur die NETID des Senders verschieden:

NETID des Testsenders	im Relaispaar?	E22 (NETID 00)
BB	ja	empfangen , -23 bis -40 dBm
07	nein	Stille , 0 von 4

Beide Sperren greifen dabei zugleich: direkt, weil `07 ≠ 00` und die Adresse kein Broadcast ist; über das Relais, weil `07` in keiner Richtung seines Paares liegt.

9.7 Das Gateway steht außerhalb

Der Rohkanal des DLOS8N hat **keine NETID**. Sie ist ein Byte innerhalb der Ebyte-Nutzlast; der SX1302 demoduliert auf der LoRa-Ebene — Frequenz, SF, Bandbreite, Syncword — und reicht die Nutzlast unverändert weiter. Er kennt das Ebyte-Protokoll nicht.

```
netid 0x07  ziel 2201  b'FREMD-3'      am Gateway empfangen,  
                                         vom E22 zugleich verworfen
```

Das Gateway ist damit kein Gruppenmitglied, sondern **passiver Mithörer** — praktisch günstig: Die Trennung wirkt zwischen den Ebyte-Knoten, während die Überwachung über MQTT beide Gruppen sieht. Nur wenn `dell/loraw_raw.py` sendet, entsteht Gruppenzugehörigkeit, und die steht dort als `EBYTE_NETID`.

9.8 Was damit nicht geht

Gezielte Einzeladressierung. Ein Rahmen an `2201` statt an den Rundruf kam im Test **nicht** an. Kapitel 4.1 des Handbuchs steht unter der Voraussetzung „in fixed-point mode“ — die Endgeräte laufen aber transparent (`REG3 = 0x80`). Im Transparentmodus wertet ein Modul das Zielfeld nur gegen Broadcast und NETID aus. Für erlaubte Paare über Einzeladressen müssten die Empfänger auf Fixpunkt (`REG3` Bit 6) umgestellt werden — der E90 läuft ohnehin schon so, weil der Relaisbetrieb es voraussetzt.